



BDBP209: Biostatistics and R programming



2024-2025



Course information and grading

- ❖ 2-credit course
- ❖ 4 hour practicals on Fridays (2PM-6PM)
- ❖ Final exam duration (2 hours)
- ❖ Two internal assessments (Dates: Feb 25, Mid March)
- ❖ Mark distribution (Final: 35, Internal: 15)
- ❖ In-class work only (so please keep up!)
- ❖ All work for each practical is due by Friday night

Evaluation policy

BDBP-209 - Biostatistics and 'R' Programming - Laboratory (JN)	
(Total number of credits = 2)	
Evaluation Component	Marks distribution
Internal Assessment - 1	7.5
Internal Assessment - 2 components	
Lab Quiz/Viva	1.5
Practical Performance	1.5
Practical Answer sheet	1.5
Lab Record	1.5
Attendance	1.5
Pre-TEE Total (PTT)	15
Term End Exam (TEE)	35
Total	50

Topics

- > Installation of R and its libraries
- > R language basics
- > Biostatistics in R including permutations and combinations, contingency tables, ROC curves, statistical parameters, confidence intervals, hypothesis testing, error analysis, ANOVA, covariance, correlation
- > Regression analysis in R
- > Clustering algorithms in R

Why R?

- > Ideal tool for biostatistics (open-source, minimal programming for biologists)
- > Huge number of statistical libraries
- > Libraries for data analytics, data mining and big data analysis (not done this semester) (>2200 libraries for bioinformatics added)
- > Efficient handling of huge amounts of data
- > www.r-project.org
- > Latest findings in statistical analysis in literature use R!

Examples of R usage in biology

- > Analysing DNA/RNA/protein sequence data
- > matrices of species community data
- > time series data
- > outputs of statistical tests for various experimental data
- > clinical data (study design, hypothesis testing, modeling disease prevalence and prediction)
- > Genomics (gene discovery, biomarker discovery, significance of alignment, differential expression of genes, proteins and metabolites)

Setup

-> Install R from the CRAN (Comprehensive R Archive Network) website, or using apt manager, or using rig and RStudio, a user-friendly IDE for R

-> If you are using only R, then one has to 'source' the file to run it.

-> R-studio is more user-friendly than R interface.

- > syntax highlighting and smart indentation

- > Integrated support for git

- > execute R code directly from editor

- > quickly jump to function definitions

- > manage multiple working directories using projects

- > Integrated R documentation and help

- > interactive debugger to diagnose and fix errors quickly

Setup steps

- `sudo apt update && sudo apt -y upgrade`
- `sudo apt -y install r-base`
- Download appropriate R-studio (Ubuntu 22, 20 etc). For eg. Ubuntu 22 (jammy)
- `wget https://download1.rstudio.org/electron/jammy/amd64/rstudio-2023.06.1-524-amd64.deb`
- `sudo apt install -f ./rstudio-2023.06.1-524-amd64.deb`

GIT and GITHUB usage in this course

- > Installation of git
- > Verify credentials of your github account so that you can sync your local codes with github
- > Upload your codes under a folder for each practical session ('Lab 1', 'Lab 2' etc)

Basic operations

-> Open R command line

> 10+2 #add

> 10-2 #subtract

> 10*2 #multiply

> 10/2 #divide

> 10^2 #exponent

Objects and functions

- > Objects are any form of input/output - letters, numbers, other things..
- > Functions are used to manipulate objects
- > Functions always have brackets, objects do not.
- > There is a function called `function()` in R! (used for creating custom functions)
- > Warning: Do not put space between the function name and the opening bracket (you will get an error)
 - > Eg. Concatenate function `c()` (Try `c(1,2,3,5)`). Output is an object, numbers 1,2,3,5 are input objects.

Object and its contents

-> `X<-'Jayashree'`

-> `print(X)`

-> `paste(X)`

-> Difference between `print()` and `paste()`: `print()` is the most generic and useful for long scripts. `paste()` converts the argument into a string.

Concatenate function c()

-> `c(1,2,3,5)`

`[1] 1 2 3 5`

-> output is an object – this object is called a vector, it has 4 elements

-> input is 4 objects/4 numbers. `c()` takes 4 objects and combines them into a new object (vector). (Think of row or column in an excel sheet as a simple example)

Math functions

-> abs(-10)

-> sqrt(10)

-> sqrt(10-1)

-> log(10)

-> log10(10)

-> exp(1)

-> sin(pi/2)

-> asin(1)

-> cos(pi)

-> acos(-1)

-> tan(0)

-> atan(0)

Function	Meaning
log(x)	log to base e of x
exp(x)	antilog of x (e^x)
log(x,n)	log to base n of x
log10(x)	log to base 10 of x
sqrt(x)	square root of x
factorial(x)	$x! = x \times (x-1) \times (x-2) \times \dots \times 3 \times 2$
choose(n,x)	binomial coefficients $n!/(x!(n-x)!)$
gamma(x)	$\Gamma(x)$, for real x ($x-1$)!, for integer x
lgamma(x)	natural log of $\Gamma(x)$
floor(x)	greatest integer less than x
ceiling(x)	smallest integer greater than x
trunc(x)	closest integer to x between x and 0, e.g. $\text{trunc}(1.5) = 1$, $\text{trunc}(-1.5) = -1$; trunc is like floor for positive values and like ceiling for negative values
round(x, digits=0)	round the value of x to an integer
signif(x, digits=6)	give x to 6 digits in scientific notation
runif(n)	generates n random numbers between 0 and 1 from a uniform distribution
cos(x)	cosine of x in radians
sin(x)	sine of x in radians
tan(x)	tangent of x in radians
acos(x), asin(x), atan(x)	inverse trigonometric transformations of real or complex numbers
acosh(x), asinh(x), atanh(x)	inverse hyperbolic trigonometric transformations of real or complex numbers
abs(x)	the absolute value of x, ignoring the minus sign if there is one

Round/truncate functions

-> `round(pi, digits=3)`

-> `floor(pi)`

-> `ceiling(pi)`

-> `signif(pi, digits=2)`

-> Understand the difference between `round()` digits argument and `signif()` digits argument

-> Try `signif(123456789, digits=2)` and `round(123456789,digits=2)`

-> `as.integer()` works as a truncating function on real numbers. `as.integer(5.7)`

Simple calculations

-> $\log(42/7.3)$

-> $5+6+3+8+$

$+ 3+2+7$

-> $2+3; 5*7; 3-7$

-> $1.2e3$ means 1200 (e3 means move the decimal pt 3 places to the right)

-> $1.2e-2$ means 0.012 (e-2 means move the decimal pt 2 places to the left)

Logical operators

-> 1>2 FALSE

-> 1<2 TRUE

-> 1<=2 #less than or equal to TRUE

-> 1==1 TRUE

-> 1==1 | 1==2 #OR

-> 1==2 & 1==1 #AND FALSE

-> 1!=1 #not equal to FALSE

-> 15421 %% 7 ==0 TRUE

Variable names and assignment

- > Variable names in R are case-sensitive, so 'y' is not the same as 'Y'
 - > Variable names should not begin with a number (for example 1x) or symbols (%x)
 - > Variable names should not contain blank spaces
- `x <- 5` (what happens if you type `x < - 5` ?)
- > Never use T or F as variable names. Use TRUE and FALSE in full always.

Group comparisons

-> 1 %in% c(1,2,3,4,5,6,7,8,9,10)

Loading libraries

- > `library(spatial)` will load the in-built library called spatial
- > `library(help=spatial)` will list the contents of the spatial library in terms of all the functions and the data sets
- > `objects(grep("spatial",search()))` will list the full contents

Installing packages in R

```
-> install.packages("tree")
```

Help

-> ?round

-> ?read.table

-> help.search("data input")

-> Other useful functions: 'find' and 'apropos' and 'example' and 'demo'

-> find("lowess") tells you what package something is in

-> apropos("lm") returns a character vector with names of all objects that match your (partial or full) enquiry

-> example(lm)

-> demo(graphics), demo(plotmath)

-> R-studio has a 'Help' menu that is easy to use

Tips

- > Know what you want to achieve
- > Keep it simple
- > Clever is good but clear is better
- > Test each line as you go along
- > Put plenty of comments (#)
- > Use self-explanatory variable and function names
- > Do not use 'attach' in programs
- > Use indentation to improve clarity of loops and if statements
- > Build programs from small, independently tested functions
- > Stop tinkering once it works effectively